WHITE PAPER OFICIAL • SDD FRAMEWORK V6.6 HARDENED EDITION

GUÍA MAESTRA DE INICIALIZACIÓN Y OPERACIÓN: SDD v6.6

Manual Operativo Paso a Paso, Comandos CLI, Pruebas de Gobernanza Pester 6 y Glosario Oficial de Términos

Guía Maestra de Inicialización y Operación: SDD Framework v6.6

> **Manual Operativo Paso a Paso para la Creación de Proyectos, Gobernanza Criptográfica HMAC-SHA256, Verificación de SLAs k6 y Generación de Expedientes de Evidencia Regulatoria Versión:** 6.6 (Regulatory & Compliance Hardened Edition)
> **Fecha:** Noviembre de 2026 **Compatibilidad:** Windows 10/11, PowerShell 5.1+, PowerShell 7+, Linux/macOS (bash/pwsh)

---

1. Introducción y Novedades de la Versión 6.6

La versión **6.6 - Regulatory Hardened Edition** endurece la gobernanza criptográfica, la integridad de especificaciones y la precisión de auditoría de SDD:

- **Candado Criptográfico HMAC-SHA256**: Sustituye el hashing básico por un HMAC-SHA256 formal con clave persistente en `.specify\.sdd_key` o mediante `$env:SDD_HMAC_KEY`.
- **Inclusión Inmutable de `plan.md` en el Sello**: Sella conjuntamente `spec.md`, `scenarios.feature` y `plan.md`. Modificar `Allowed Files` tras la aprobación humana rompe el candado inmediatamente.
- **Aprobación Human-in-the-Loop (HITL) Estricta**: Queda prohibida la auto-aprobación del agente. Toda aprobación formal captura obligatoriamente la identidad del firmante (`user.name <email>`), fecha UTC y algoritmo.
- **Paridad BDD 1:1 en Plantillas**: La plantilla inicial incluye escenarios para `@AC-1.1` hasta `@AC-1.4`, garantizando paridad 100% desde la creación.
- **Cero Falsos Positivos en Gate 1**: Se excluye `tests/performance/k6_benchmark.js` de las alertas de Spec Drift.
- **Hook PreToolUse Fail-Closed**: Bloqueo físico en tiempo real de escrituras en `src/` si no hay especificación aprobada o si se detecta manipulación de archivos.
- **Transparencia en Gate 9 (k6)**: Estado real `SKIPPED_LOCAL` con advertencia `WARN` si k6 no está instalado localmente, delegando la medición activa al runner de CI/CD sin falsear certificaciones.
- **Soporte Nativo para Java (Maven/Gradle)**: Nuevo preset `java-maven` en `sdd.ps1 -Action configure-stack`.
- **Alineación del Lenguaje Regulatorio**: Generación de "Evidencia Técnica Auditable" para ISO 27001, FDA 21 CFR Part 11, SOC 2 y HIPAA.
- **Suite de Pruebas de Gobernanza (Pester 6)**: Validación automatizada mediante `tests/v66-governance.Tests.ps1`.

---

2. Requisitos Previos e Instaladores Disponibles

Requisitos del Sistema

1. **PowerShell**: Versión 5.1 (incluida en Windows 10/11) o PowerShell Core 7+.
2. **Git**: Versión 2.30 o superior en el PATH del sistema (`git config user.name` y `user.email` configurados).
3. **Navegador Web para Exportación PDF (Edge o Chrome)**: - Microsoft Edge (preinstalado en Windows) o Google Chrome. - En su defecto, se emite HTML optimizado para `@media print` (Ctrl+P).
4. **k6 (Opcional en host local)**: Para ejecución en vivo de benchmarks de carga.
5. **Pester 6 / PScriptAnalyzer (Opcional)**: Disponibles localmente en el directorio `tools/` .

Scripts de Inicialización Disponibles

- `setup-sdd-project-v66.ps1` : Inicializador nativo en PowerShell.
- `setup-sdd-project-v66.py` : Inicializador multiplataforma en Python.

---

3. Guía Paso a Paso para Inicializar un Proyecto

Paso 1: Ejecutar el Inicializador

Abra una consola de PowerShell en la carpeta raíz deseada y ejecute:

```
powershell -ExecutionPolicy Bypass -File .\setup-sdd-project-v66.ps1 -ProjectName "MiSistemaSeguro" -Stack "p
```

Parámetros admitidos:

- `-ProjectName <string>` : Nombre del directorio de trabajo a crear.
- `-Stack <string>` : Stack tecnológico (`python` , `typescript` , `csharp` , `golang` , `rust` , `flutter` , `java`).
- `-DestinationPath <string>` : Ruta de destino personalizada (opcional).
- `-GitInit` : Inicializa automáticamente un repositorio Git y configura el hook pre-commit.
- `-AutoLaunch <auto|ide|cli|none>` : Controla el lanzamiento del agente Antigravity tras la inicialización.
- `-NoAutoLaunch` : Desactiva el lanzamiento automático interactivo (ideal para CI/CD o scripts de automatización).

Paso 2: Verificar la Estructura Generada

```
MiSistemaSeguro/
├── .agents/
│   ├── hooks.json               # Hooks SessionStart, PreToolUse (Fail-Closed) y PostToolUse
│   └── skills/                  # 20 Skills Canonicas de SDD v6.6
├── .github/workflows/sdd-gate.yml# Pipeline GitHub Actions con 9 Aduanas
├── .gitlab-ci.yml               # Pipeline GitLab CI
├── .specify/
│   ├── .sdd_key                 # Clave persistente HMAC-SHA256 (256-bit)
│   ├── harness-config.json      # Configuracion del proyecto y Allowed Files
│   └── templates/               # Plantillas EARS, BDD (AC-1.1..1.4) y plan
├── harness/
│   ├── sdd.ps1                  # Comando CLI unificado del arnes
│   ├── spec-gate.ps1            # Auditor Nonuplo (9 Hard Gates)
│   ├── verify-security.ps1      # Verificador SAST y Secretos (Aduana 8)
│   ├── verify-performance.ps1   # Verificador de Rendimiento k6 (Aduana 9)
│   └── export-compliance.ps1    # Generador de Expediente Regulatorio PDF/HTML
├── progress/                    # Expedientes PDF, reportes y dashboards
├── specs/                       # Especificaciones EARS y BDD
├── tests/
│   ├── performance/             # Benchmarks de carga k6
│   └── v66-governance.Tests.ps1 # Pruebas de gobernanza Pester 6
├── CHECKPOINTS.md               # Bitacora inmutable de firmas criptograficas
└── feature_list.json            # Backlog formal con hashes HMAC e identidad
```

---

4. Guía Operativa del Ciclo de Vida SDD v6.6

4.1. Creación de una Funcionalidad

```
.\harness\sdd.ps1 -Action create -FeatureId "FEAT-001" -FeatureName "Autenticacion Segura"
```

Esto genera automáticamente: - `specs/FEAT-001/spec.md` : Plantilla EARS con criterios `AC-1.1` a `AC-1.4` . - `specs/FEAT-001/scenarios.feature` : Plantilla Gherkin con escenarios `@AC-1.1` a `@AC-1.4` . - `specs/FEAT-001/plan.md` : Arquitectura técnica y lista explícita de `## Allowed Files` . - `specs/FEAT-001/tasks.md` : Desglose secuencial de tareas.

4.2. Aprobación Formal Human-in-the-Loop (HITL)

Una vez revisada y completada la especificación con el agente, el desarrollador humano aprueba el alcance:

```
# En consola interactiva (solicita confirmacion humana S/N)
.\harness\sdd.ps1 -Action approve -FeatureId "FEAT-001" -Reason "Aprobacion inicial de arquitectura"

# En entornos de automatizacion o CI/CD
.\harness\sdd.ps1 -Action approve -FeatureId "FEAT-001" -Approver "DevOps Engineer <devops@empresa.com>" -For
```

Resultado: - Se genera el sello criptográfico **HMAC-SHA256** protegiendo `spec.md` , `scenarios.feature` y `plan.md` . - Se registra la identidad del firmante y la marca de tiempo UTC en `feature_list.json` y `CHECKPOINTS.md` . - El estado pasa a `in_progress` , desbloqueando el permiso de escritura en `src/` .

4.3. Implementación y Verificación de Ciberseguridad

```
# Escaneo de secretos y analisis SAST (Aduana 8)
.\harness\sdd.ps1 -Action verify-security
```

4.4. Verificación de Rendimiento y SLAs (k6)

```
# Ejecuta medicion k6 si esta instalado, o emite estado transparente SKIPPED_LOCAL
.\harness\sdd.ps1 -Action verify-performance -FeatureId "FEAT-001"
```

4.5. Auditoría Nónupla y Cierre Formal

```
# Evalua las 9 compuertas inquebrantables de spec-gate.ps1
.\harness\sdd.ps1 -Action complete -FeatureId "FEAT-001"
```

Si todas las aduanas son superadas con cero violaciones: - Se emite el **Completion Seal**. - El estado de la funcionalidad pasa a `done`.

4.6. Exportación del Expediente de Evidencia Regulatoria

```
# Exporta el expediente completo en PDF oficial de alta resolucion
.\harness\sdd.ps1 -Action export-compliance -Format pdf

# Exporta en todos los formatos (PDF, HTML, MD) y abre el documento resultante
.\harness\sdd.ps1 -Action export-compliance -Format all -Open
```

Los artefactos se depositan en `progress/compliance_dossier.pdf` y `progress/compliance_dossier.html`.

---

5. Ejecución de la Suite de Pruebas de Gobernanza (Pester 6)

Para auditar y verificar que los controles de gobernanza, el candado HMAC y las aduanas del framework operan con integridad:

```
powershell -ExecutionPolicy Bypass -Command "& { Import-Module '.\tools\Pester\6.2.0\Pester.psd1' -Force; Inv
```

Esta suite valida de forma automatizada: 1. Candado HMAC-SHA256 y persistencia de llave en `.specify\.sdd_key`. 2. Detección instantánea de *Spec Tampering* al modificar `plan.md` o `spec.md`. 3. Paridad BDD 1:1 en la plantilla de funcionalidades. 4. Ausencia de falsos positivos en Gate 1 sobre `k6_benchmark.js`. 5. Reporte verídico `SKIPPED_LOCAL` en Gate 9 cuando k6 está ausente. 6. Configuración exitosa del stack Java (`java-maven`).

---

6. Glosario Oficial de Términos y Acrónimos SDD v6.6

6.1. Tabla de Referencia Rápida (A–Z)

Término / Sigla	Nombre Completo / Artefacto	Categoría	Definición Operativa en SDD v6.6
`AC-x.x`	Acceptance Criterion	Requisitos	Identificador de criterio de aceptación en `spec.md` que exige paridad 1:1 en `scenarios.feature` (`@AC-x.x`).
`Allowed Files`	Lista Blanca de Archivos Permitidos	Gobernanza	Sección en `plan.md` que declara las únicas rutas que pueden crearse o modificarse; auditada por Gate 1 y `PreToolUse` .
`approval_hash`	Sello Criptográfico de Aprobación	Criptografía	Firma `HMAC-SHA256` calculada sobre la tríada normalizada `spec.md + scenarios.feature + plan.md` .
`BDD`	Behavior-Driven Development	Metodología	Especificación ejecutable redactada en lenguaje Gherkin (`Given-When-Then`) dentro de `scenarios.feature` .
`CHECKPOINTS.md`	Bitácora Inmutable de Firmas	Auditoría	Registro histórico cronológico de sellos HMAC, cierres de funcionalidades e identidad de aprobadores.
`Clean Architecture`	Arquitectura Limpia Concéntrica	Arquitectura	Separación del código en 4 anillos (`domain` , `application` , `presentation` , `infrastructure`) con dependencia hacia el centro.
`completion_hash`	Sello Criptográfico de Cierre	Criptografía	Hash definitivo emitido por `sdd.ps1 -Action complete` tras superar las 9+1 compuertas duras.
`CSPRNG`	Cryptographically Secure PRNG	Criptografía	Generador aleatorio criptográfico utilizado para crear la clave secreta de 256 bits en `.specify/.sdd_key` .
`DAG`	Directed Acyclic Graph	Planificación	Grafo acíclico dirigido que modela las dependencias (`depends_on`) entre funcionalidades en `feature_list.json` .

Término / Sigla	Nombre Completo / Artefacto	Categoría	Definición Operativa en SDD v6.6
`Design Tokens`	`.specify/design-system/design-tokens.json`	UI/UX	Variables atómicas de color, tipografía y espaciado validadas con contraste matemático WCAG 2.1 AA ($\geq 4.5:1$).
`Double-Loop TDD`	Desarrollo Guiado por Pruebas de Doble Bucle	Calidad	Bucle externo de pruebas de aceptación BDD combinado con bucle interno de pruebas unitarias de dominio.
`EARS`	Easy Approach to Requirements Syntax	Requisitos	Sintaxis formal de 5 cláusulas (`Ubiquitous` , `Event-driven` , `State-driven` , `Optional` , `Unwanted`) libre de ambigüedades.
`El Auditor Nónuplo`	`harness/spec-gate.ps1`	Calidad	Motor determinista que evalúa la Compuerta 0 (HMAC) más las 9 Compuertas Duras antes de autorizar el cierre.
`Evidencia Técnica Auditable`	`progress/compliance_dossier.pdf`	Cumplimiento	Expediente técnico determinista generado para soportar auditorías ISO/IEC 27001, FDA 21 CFR Part 11, SOC 2 e HIPAA.
`Fail-Closed`	Denegación por Defecto (`pre-tool-enforce.ps1`)	Seguridad	Política del hook `PreToolUse` que bloquea cualquier escritura en `src/` si falta el sello HMAC o si `plan.md` fue alterado.
`Fake Test`	Prueba Falsa o Complaciente	Calidad	Antipatrón (`assert True == True` , cuerpo `pass` , `@skip`) bloqueado físicamente por la Compuerta 5.
`HITL`	Human-in-the-Loop	Gobernanza	Principio que exige firma humana explícita (`approved_by` , `approved_at`) y prohíbe la auto-aprobación por agentes de IA.
`HMAC-SHA256`	Hash-based Message Authentication Code	Criptografía	Algoritmo criptográfico simétrico (RFC 2104) con clave de 256 bits que protege la tríada documental contra adulteraciones.
`k6 Benchmark`	`tests/performance/k6_benchmark.js`	Rendimiento	Script declarativo de pruebas de carga que audita latencias $P_{95} \leq 200\ \text{ms}$, $P_{99} \leq 500\ \text{ms}$ y tasa de error 0%.
`SAST`	Static Application Security Testing	Seguridad	Escaneo estático de código fuente (`verify-security.ps1`) para detectar inyecciones OWASP y secretos expuestos.

Término / Sigla	Nombre Completo / Artefacto	Categoría	Definición Operativa en SDD v6.6
`SKIPPED_LOCAL`	Omisión Local Transparente (Gate 9)	Rendimiento	Estado reportado cuando el binario `k6` no está instalado localmente, validando el contrato y delegando la medición activa a CI/CD.
`Spec Drift`	Deriva de Alcance	Gobernanza	Modificación de archivos fuera de la lista `Allowed Files` o adición de código no respaldado por `spec.md`.
`Spec Tampering`	Adulteración de Especificación o Plan	Criptografía	Modificación no autorizada de `spec.md`, `scenarios.feature` o `plan.md` posterior al sellado HMAC; bloquea Gate 0.
`STRIDE`	Modelo de Amenazas de 6 Vectores	Seguridad	Análisis obligatorio en `plan.md` (*Spoofing, Tampering, Repudiation, Information Disclosure, DoS, Elevation of Privilege*).
`TopoSort`	Ordenamiento Topológico	Observabilidad	Algoritmo que ordena linealmente el grafo DAG en el Cockpit Web Dashboard (`progress/sdd_dashboard.html`).
`Tríada Letal`	Lethal Triad	Seguridad	Confluencia de alto riesgo en un mismo módulo: *Datos Sensibles + Entrada Externa No Confiable + Salida a Red*.